

IDRIB - ICU Discharge Risk & Intervention Bot

Final Bot Implementation Report

Jinalkumari Vaniya | Hofstra University | May 2026

Abstract

This report documents the final bot layer for IDRIB, a Streamlit-based front-end prototype that presents ICU discharge risk results in a structured clinical workflow. The bot does not function as a general-purpose chatbot. Instead, it acts as a controlled interface around the project's saved scoring outputs, allowing patients and clinicians to interact with the risk model through predefined views, decision rules, and logs. The implementation uses local files produced by the project pipeline, including the trained model, feature list, imputer, scored CSV outputs, and saved charts. The report describes the bot files, the runtime pipeline, the patient and clinician interfaces, the escalation and warning logic, the JSON contract, and the visual evidence captured in the Streamlit screenshots.

The purpose of the bot is to turn a readmission score into an operational workflow. For patients, the interface asks a closed set of follow-up questions, detects urgent patterns, and either ends the conversation cleanly or escalates to the care team. For clinicians, the dashboard exposes high-risk cases, explanation cues, and a chat log for review. In that way, the bot makes the scoring engine usable without changing the underlying model.

1. Introduction and Project Scope

IDRIB stands for ICU Discharge Risk and Intervention Bot. The bot layer was built after the scoring engine had already been completed, so its role is not to retrain the model or invent new clinical logic. Its job is to package the existing output of the project into a patient-facing and clinician-facing interface that can be run locally in Streamlit. The design follows the project documents supplied with the notebook: a scoring engine that produces a 0 to 100 readmission score, a tier assignment, a list of top drivers, and a tier-based intervention recommendation.

The bot is intentionally narrow in scope. It supports the specific discharge workflow. the Progress Dashboard summarizes the state of the build, the Patient Bot conducts a structured check-in, the Clinician Dashboard highlights risk and review information, the Workflow View explains the operational sequence, and the Files and JSON Contract page documents the inputs and outputs required to integrate the bot with the scoring engine. This report describes each of those parts as they are implemented in the current project files.

2. Files and Runtime Structure

The current implementation uses a small, explicit file structure. The Streamlit app loads local artifacts at runtime, and the screenshots show the same files being referenced in the interface. This separation is important because the bot does not need to rebuild the model each time it runs. It only needs to load the saved outputs and present them in a readable format.

The runtime folder structure in the application is simple: the app expects a model's folder, a csv folder, a charts folder, and a logs folder. The bot uses these folders directly, which makes the implementation easy to inspect and easy to reproduce for a class project or demo presentation.

File	Role in the bot system
models/risk_model.pkl	Serialized classifier loaded with joblib and used to produce the readmission score.
models/feature_cols.pkl	Ordered list of the 39 feature columns expected by the model.
models/imputer.pkl	Median imputer used to handle missing values consistently.
csv/features_matrix.csv	Processed feature matrix used to support the demo and validate the pipeline.
csv/patient_risk_scores.csv	Scored output file that powers the dashboard alerts and clinician review.
charts/	Folder containing saved evaluation charts used in the progress dashboard.
logs/patient_chat_log.csv	Conversation log written by the patient bot for clinician review.
bot-demo.py	Main Streamlit application that renders all pages and controls the conversation flow.

3. Bot Pipeline and Application Architecture

The bot pipeline is a wrapper around the scoring engine, not an independent prediction system. When the Streamlit app starts, it loads the serialized model, the feature list, and the imputer. It also loads the scored CSV file so that the dashboard can display real project outputs instead of placeholder text. From there, the user can switch between views that represent different parts of the discharge workflow.

1. Load local artifacts from models/ and csv/.
2. Render the selected Streamlit view.
3. For the patient bot, step through a fixed set of questions using button choices.
4. Apply deterministic rules for escalation, completion, and follow-up reminders.
5. Write each interaction to the chat log for clinician review.
6. Expose the output in the clinician dashboard and the JSON contract view.

This pipeline matches the project's bot specification. The scoring engine remains the source of truth for risk, while the bot is the presentation and orchestration layer. That distinction matters because it keeps the app auditable: the user can see the score, the tier, the written intervention, and the chat history without the bot improvising clinical advice.

4. Patient-Facing Interface Design

The patient-facing page is built as a closed conversation. The bot begins with a medication question and then advances through a small sequence of structured responses. The patient does not type into a free-text box. Instead, the interface uses buttons for the available answers, which keeps the flow consistent and avoids ambiguous input. This design choice is visible in the screenshots and in the code: each step offers a fixed set of choices, and the next bot message depends on the selected option.

The conversation asks about medication adherence, breathing, symptoms, and appointment status. The wording is simple because the goal is triage, not diagnosis. If the patient selects an urgent option, the bot stops the script and escalates. If the conversation reaches a normal end state, the bot closes with a completion message. This makes the interface feel structured and clinically bounded rather than conversational in the general chatbot sense.

- Medication adherence is used as the first branch point.
- Breathing is scored on a 1 to 5 button scale.
- Symptoms are captured as no symptoms, mild symptoms, or urgent symptoms.
- Follow-up appointment status is captured as yes, no, or not sure.
- The conversation ends when the check-in is complete or when escalation is triggered.

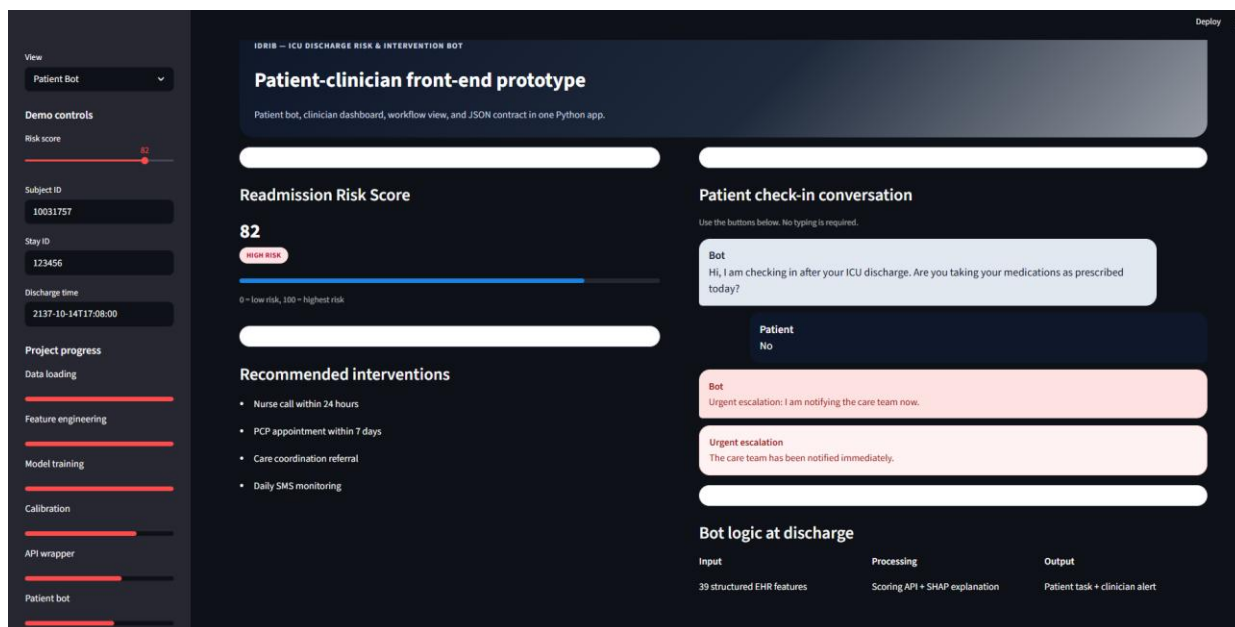


Figure 1. Patient bot with urgent escalation. The interface shows the conversation after the patient selects a high-risk response, and the message is styled in red to signal immediate attention.

This screen demonstrates the escalation behavior. The red bot message is used only when the patient response indicates urgent concern. The conversation ends immediately after the escalation message, which prevents additional questions from being asked when the care team needs to be notified. That behavior is important because the bot must stop the script rather than continue a routine check-in when urgent conditions are detected.

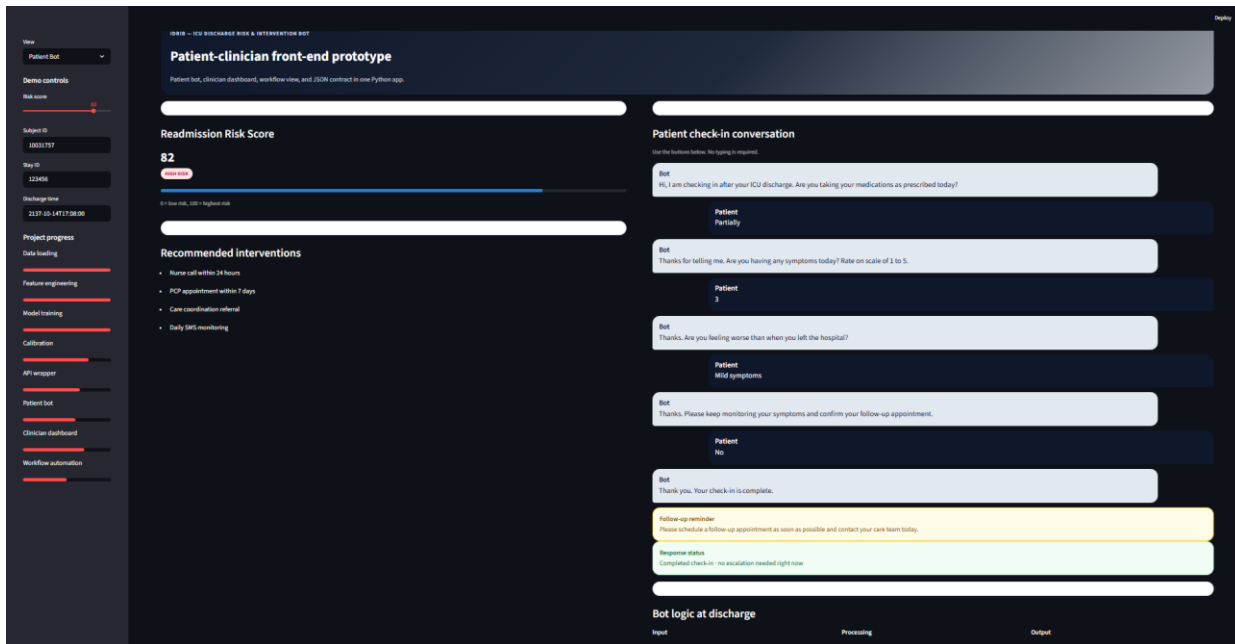


Figure 2. Patient bot showing the urgent escalation ending state. Once the red condition is met, the bot confirms that the care team has been notified and no further questions are asked.

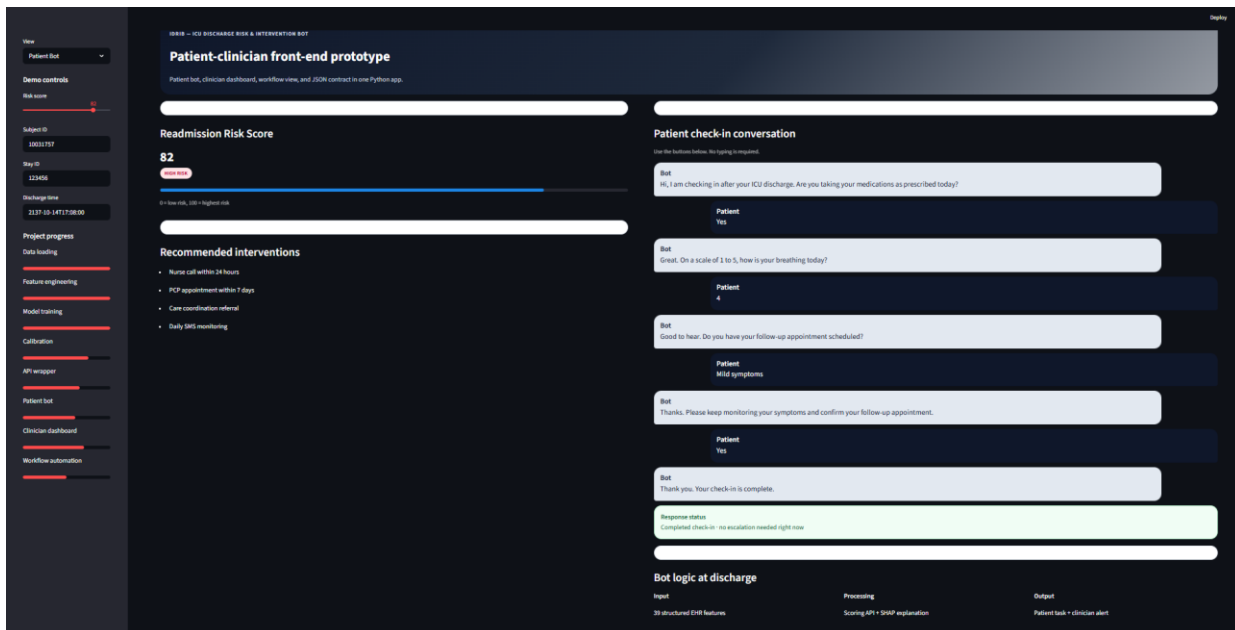


Figure 3. Patient bot showing a normal progression through the script and the follow-up reminder panel. The yellow box appears when follow-up action is needed but the situation is not an urgent emergency.

This screen shows the middle of the patient flow. The bot has already moved through medication and breathing questions, and the response path results in a yellow follow-up reminder rather than an immediate escalation. The reminder encourages the patient to schedule follow-up and contact the care team, which is a more moderate response than the red urgent alert.

This final patient screen is the clean stop state for the script. The user can see that the escalation has been triggered and that the conversation has ended. This is the safest behavior for a discharge follow-up workflow because it prevents the bot from continuing with routine questions after an urgent issue has already been identified.

5. Escalation, Warning, and Completion Rules

The bot has three visible response states. The first is the normal completion state, which appears when the script ends without a concerning response. The second is the yellow warning state, which is used when the patient does not report an immediate emergency but does show signs that follow-up action is needed. The third is the red escalation state, which is used when the response indicates urgent symptoms or a low breathing score. These states are clearly distinguished in the interface so the user can immediately understand the level of concern.

The yellow warning logic is specifically tied to moderate concern. In the current implementation, it is shown when breathing is below the safer range and the patient has mild or urgent symptoms together with no or uncertain follow-up appointment status. The purpose is to urge the patient to schedule an appointment and contact the care team without implying that the situation is already a medical emergency.

The red escalation logic is stricter. It is triggered by urgent symptoms, a low breathing score, or a clearly unsafe response in the medication branch. When that occurs, the script stops asking new questions. The bot then shows a red alert and states that the care team has been notified. This is the safest behavior for a triage-style bot because it removes any ambiguity about the next step.

These states are not cosmetic only; they are part of the workflow. A normal completion message means the check-in can stop. A yellow warning means the patient should be nudged toward follow-up. A red alert means the conversation must end and the care team must be notified. The interface therefore supports both clarity and safety.

6. Clinician-Facing Interface Design

The clinician-facing page is designed to make the model output useful at the point of care. It shows a list of high-risk discharges, the reasons the patient is considered high risk, the workflow status, and the log of recent patient messages. Unlike the patient page, this view is meant for review and oversight. It is where a clinician can verify why a patient was flagged and check what happened in the follow-up conversation.

The dashboard is intentionally concise. It summarizes the score, tier, and top risk drivers rather than presenting the full underlying model object. This is consistent with the project design: the clinician needs interpretation and action cues, not raw technical internals. The risk summary and the conversation log together give the reviewer enough information to decide whether outreach, case management, or additional evaluation is needed.

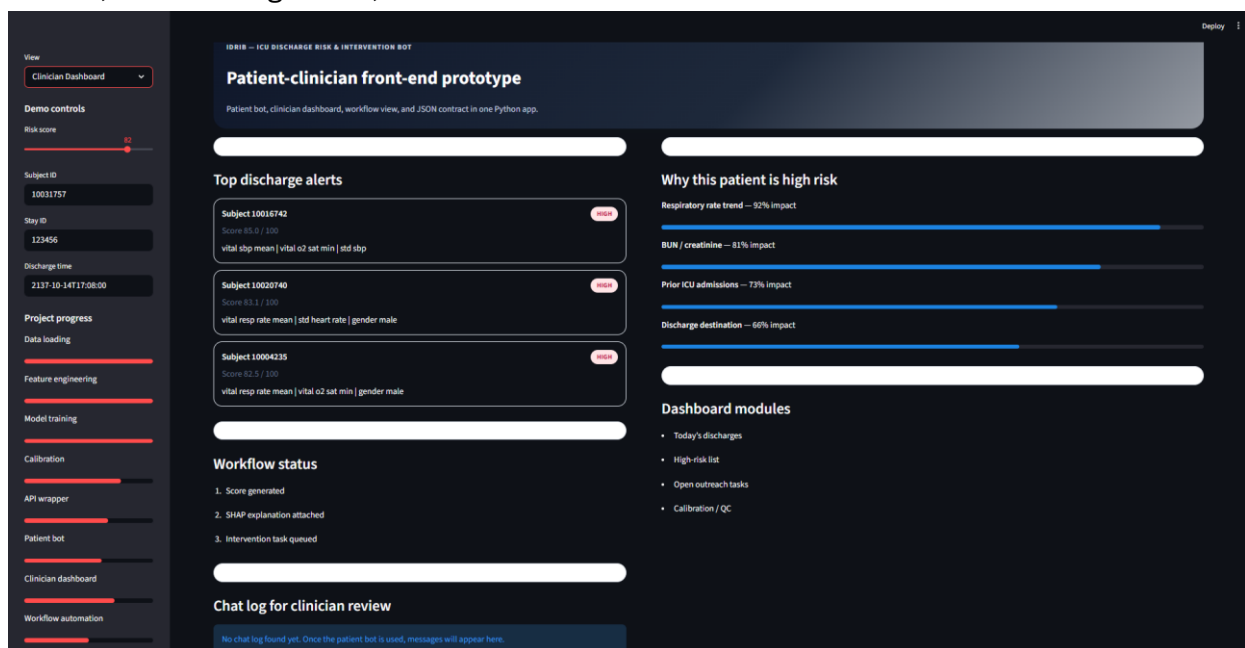


Figure 4. Clinician dashboard showing high-risk alerts, SHAP-driven explanations, workflow status, and the patient chat log review panel.

The clinician dashboard screenshot shows the core review elements in one place. The high-risk cards make it easy to scan the most important patients first. The reasons section helps the clinician see why the score was high. The chat log section adds accountability by preserving the conversation, so a clinician can confirm what the patient reported and how the bot responded.

7. JSON Contract and Interaction Logging

The Files and JSON Contract page documents how the bot can be integrated with the scoring layer. The request payload includes the patient identifiers, the discharge time, and the structured feature dictionary. The response payload returns the risk score, tier, probability, top drivers, and intervention list. This keeps the interface predictable and makes it possible to connect the bot to other systems later if needed.

In the current implementation, the JSON contract is shown as a readable example rather than as a live API endpoint. That is appropriate for an academic prototype because the goal is to demonstrate the shape of the exchange and the file responsibilities. The code and the screenshot both show the file names that would be required for a future service wrapper.

Every patient message and bot reply is also written to a CSV log in the logs folder. The log stores timestamp, session identifier, patient and stay identifiers, the step number, the role, the text, and whether the conversation ended in escalation or completion. That makes the bot easier to review during discharge rounds and supports auditability for future evaluation.

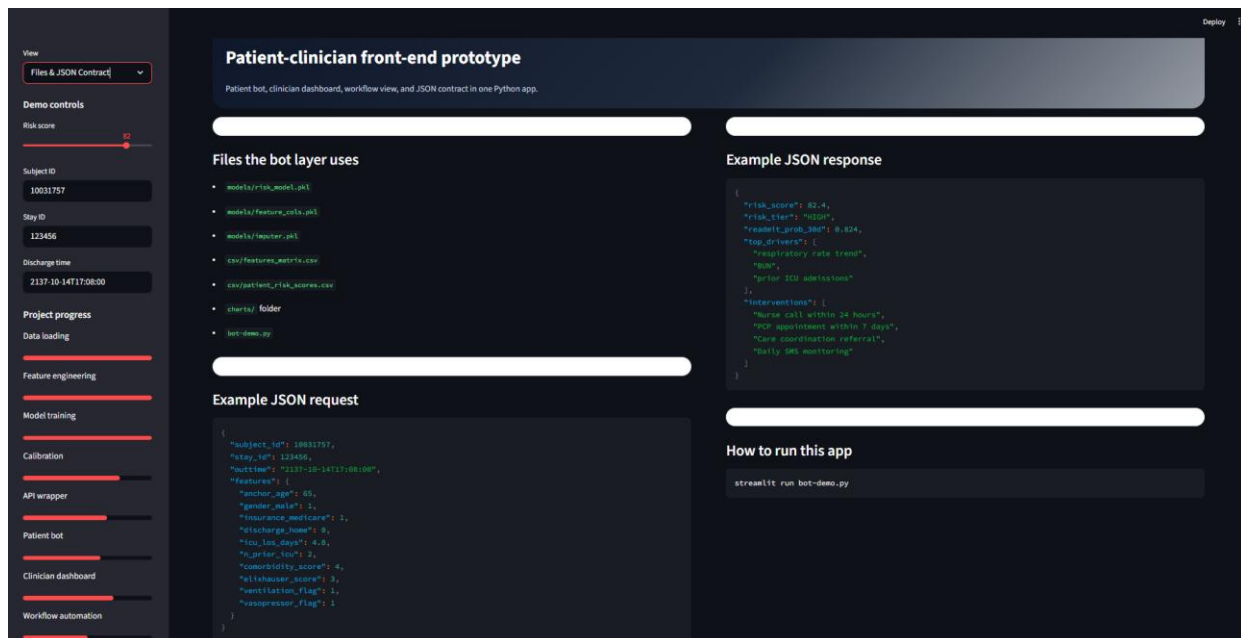


Figure 7. Files and JSON contract view. The page lists the local model artifacts, the feature matrix, the scored output file, and example request and response payloads.

This screen is important because it ties the user interface back to the file structure. The bot is not operating on hidden state. It is explicitly built around named files, which makes the implementation easy to verify and reuse. The sample JSON request and response also show the type of structured interface the project expects.

8. Workflow View and Progress Dashboard

The Workflow View explains the overall sequence of the bot. It shows the discharge event, the model scoring step, and the task generation step, followed by a summary of the files that support the flow. This view is useful in an academic setting because it connects the code, the outputs, and the operational use case into one picture. The workflow is simple by design: it takes the scoring result and turns it into a usable follow-up process.

The Progress Dashboard shows how the project is organized and how far each component has progressed. The build completion bars summarize the major phases of development. The risk tier chart shows the distribution of scored patients. The roadmap and validation table provide a quick summary of the project status. This is the right view for presenting progress to an instructor because it communicates both implementation maturity and output structure at a glance.

The progress dashboard is not a generic reporting page. It is tied to the specific local outputs of the project, including the scored CSV and the saved charts. Because of that, it functions as a project summary inside the app itself. The design makes it easy to see that the bot is built from actual project artifacts rather than placeholder content.

This second dashboard screenshot shows the charts in a more complete form. It reinforces that the application is reading local files and presenting them in a structured dashboard rather than constructing charts on the fly from unknown sources. For a final report, this kind of screenshot is useful because it links the written explanation to the live interface output.

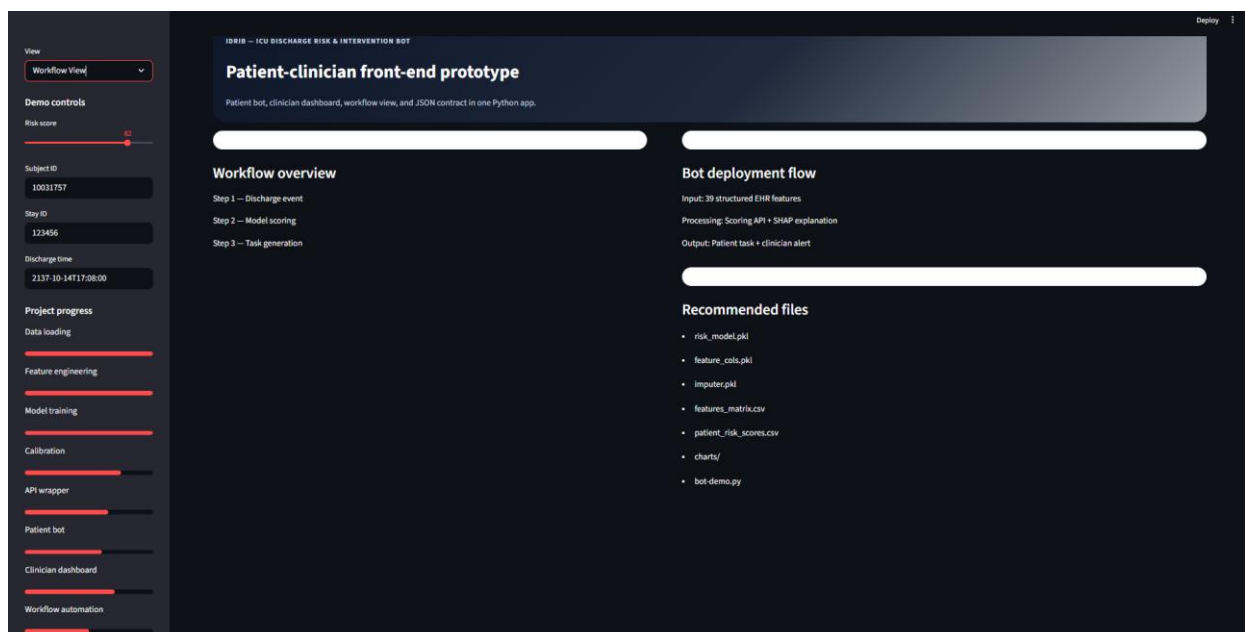


Figure 8. Workflow View showing the discharge sequence and the recommended files that support the bot runtime.



Figure 10. Expanded progress dashboard with charts and validation table. This version shows the same build status but provides a fuller visualization of the saved outputs.

9. How the Bot Helps Patients and Clinicians

For patients, the bot offers a simple and structured way to complete discharge follow-up. It reduces confusion by using fixed buttons instead of open-ended text entry. It also gives the patient a clear signal about what to do next when a warning or escalation appears. In the best case, it confirms that the check-in is complete. In the more serious cases, it directs attention toward immediate follow-up or care-team review.

For clinicians, the bot improves visibility. It surfaces which discharges are high risk, why they are high risk, and what action path is attached to the score. The chat log adds a review trail, and the dashboard makes it possible to scan recent outcomes quickly. That means the bot supports discharge planning, outreach prioritization, and follow-up coordination without replacing clinician judgment.

The biggest practical value of the bot is that it turns model output into action. A score on its own is not enough. The interface translates the score into a patient conversation, a tier, a reminder, an escalation, and a clinician review record. That conversion from prediction to workflow is what makes the bot useful as a health informatics prototype.

10. Conclusion

The final IDRIB bot is a polished Streamlit prototype built from the project's saved files and outputs. It is grounded in the scoring engine, but it adds the layers needed for practical use: a patient script, a clinician review view, a warning and escalation system, a file-based JSON contract, and persistent logging. The resulting application is understandable, auditable, and consistent with the project materials that were supplied in the notebook and supporting documents.

Because the bot is built around local files and deterministic rules, it is appropriate for academic demonstration and future extension. It shows how a machine learning result can become a structured interface that helps patients stay engaged after ICU discharge and helps clinicians focus their attention on the highest-risk cases first.